

UNIVERSIDAD ESTATAL A DISTANCIA

ESCUELA DE CIENCIAS EXACTAS Y NATURALES

CARRERA INGENIERÍA INFORMÁTICA

CÁTEDRA DESARROLLO DE SISTEMAS

PROYECTO

Asignatura: SISTEMAS OPERATIVOS

CÓDIGO: 00881

SEGUNDO CUATRIMESTRE 2025

Nombre completo: Héctor Andrés Arrieta Sancho

Número de identificación: 114070263

Centro Universitario: Heredia 34

Grupo: 03

Fecha: 9/08/2025

Tabla de contenido

Introducción.....	3
Marco teórico.....	4
Desarrollo	7
Descripción del programa.....	7
Como funciona (Guía)	9
Limitaciones.....	15
Conclusiones.....	17
Bibliografía	19

Tabla de figuras

Ilustración 1, Captura Windows.....	12
Ilustración 2, Captura Linux.....	13
Ilustración 3, Captura instalación	14

Introducción

El presente trabajo tendrá como objetivo mostrar el desarrollo y funcionamiento de una aplicación de escritorio multiplataforma creada en Java, diseñada para ejecutarse en sistemas operativos Windows y Linux. La investigación presentará los conceptos necesarios para comprender cómo se ejecuta este tipo de programas, así como la forma en que una misma aplicación puede mantener su funcionalidad en distintos entornos.

La estructura del trabajo incluirá un marco teórico con los conceptos fundamentales relacionados con sistemas operativos y ejecución multiplataforma, un desarrollo donde se describirá paso a paso la aplicación y sus funcionalidades, una sección de limitaciones para señalar aspectos a mejorar. Con este recorrido, la investigación permitirá comprender de manera clara cómo se implementa y utiliza una aplicación multiplataforma en un entorno académico simulado.

Se explicará la diferencia entre los tipos de aplicaciones según su forma de distribución y uso, destacando las que requieren instalación y las que pueden ejecutarse directamente como archivo JAR, siempre que el equipo cuente con el entorno Java instalado. Esta distinción permitirá comprender las posibilidades de instalación y ejecución que ofrece la aplicación desarrollada.

La parte práctica se centrará en la aplicación Cursos Académicos UNED, la cual simulará un entorno académico básico. A través de ella, se podrá registrar estudiantes, iniciar sesión con doble verificación, crear cursos con control de estado, asignar materias de forma limitada y consultar información. Se mostrarán las distintas formas de ejecutar el programa: como proyecto editable en NetBeans, mediante el archivo JAR que permite su apertura en Windows y Linux, o a través de un instalador gráfico en Windows que crea un acceso directo en el escritorio.

Marco teórico

El proyecto se apoya en ideas que permiten comprender por qué la aplicación funciona como lo hace y por qué las decisiones tomadas tienen sentido en un entorno de un software académico que se verá a continuación.

Un sistema operativo: es el programa que coordina el uso del equipo y sirve de puente entre las aplicaciones y los recursos físicos. Su tarea es organizar la ejecución de programas, administrar la memoria y los archivos, y ofrecer una forma segura de acceder a la información. Sobre esa base, la aplicación desarrollada reproduce procesos comunes que se esperan en un ambiente controlado: ingreso con verificación, registro ordenado de datos y consulta confiable de lo almacenado.

“Los sistemas operativos son sistemas de software que hacen que el uso de computadoras sea más cómodo para los usuarios, desarrolladores de aplicaciones y administradores de sistemas” (Deitel & Harvey Deitel, 2021, p. 14)

La aplicación de escritorio multiplataforma: se apoya en Java para poder ejecutarse en diferentes sistemas, en este caso Windows y Linux. La portabilidad se logra porque el programa se empaqueta en un archivo ejecutable de Java (JAR) y necesita el entorno de Java instalado (JDK) para funcionar. Al contar con ese requisito, el mismo archivo puede abrirse en ambos sistemas sin cambios en el código. Esta decisión favorece la entrega del proyecto y la demostración de su funcionamiento en distintos equipos.

“Java permite crear programas ejecutables en una gran variedad de sistemas y dispositivos controlados por computadora. Esto es llamado “escribir una vez para ejecutar en cualquier parte”.” (Deitel & Harvey Deitel, 2021, p. 18)

El almacenamiento de datos: se resuelve con archivos de texto organizados por carpetas. Esta elección tiene dos propósitos: simplificar la ejecución (no requiere instalar un servidor de base de datos) y mostrar de manera transparente dónde queda la información. Guardar en archivos permite comprender con facilidad el flujo de datos: el estudiante se registra y el sistema genera un archivo con sus datos; el curso se crea y queda anotado; la asignación vincula ambos y se guarda para su posterior consulta. A la vez, esta decisión introduce una

reflexión sobre seguridad y escalabilidad: un archivo puede abrirse y editarse sin control, y por eso conviene pensar en medidas de resguardo, como cifrar contraseñas, validar integridad y, en escenarios más exigentes, migrar hacia una base de datos con controles más finos.

El ingreso con doble verificación: este segundo paso agrega una barrera adicional ante intentos de acceso indebido. La idea de doble verificación nace del principio de no depender de un único dato que, si se expone, deja la cuenta desprotegida. En entornos reales existen variantes más avanzadas, como códigos temporales en aplicaciones de autenticación o envíos por correo o teléfono. En el proyecto, la pregunta secreta permite cumplir el requerimiento y, al mismo tiempo, ilustrar el valor de una segunda comprobación.

“La autenticación de dos factores proporciona una capa secundaria de seguridad que hace que sea más difícil para los piratas informáticos acceder a los dispositivos y las cuentas en línea de una persona para robar información personal. Con la autenticación de dos factores habilitada” (Reyes Riveros, Mendoza de los Santos, & Salinas Meza, 2023, p. 89)

La gestión de estudiantes y cursos: responde a un proceso natural en un entorno académico. Primero se registra a la persona estudiante con los datos indispensables para identificarla y contactarla. Luego se crea el catálogo de cursos y se marca su estado como activo o inactivo. Esa marca de estado es decisiva, porque determina si un curso puede ser elegido o no durante la matrícula. Por último, se permite que el estudiante seleccione sus cursos con un límite definido, lo que previene sobrecargas y mantiene un orden mínimo en el registro. Este encadenamiento de acciones refleja reglas de uso conocidas en instituciones educativas y ayuda a que la experiencia sea predecible.

La separación de responsabilidades: en el código simplifica el mantenimiento y mejora la claridad. Esta distribución evita mezclar la presentación con la lógica y permite que cada parte cumpla un papel concreto. Gracias a ello, cuando se cambia una regla, no es necesario reescribir toda la interfaz, y cuando se ajusta la forma de guardar, no se altera lo que ve la persona usuaria.

El control de roles: distingue lo que puede hacer el administrador de lo que puede hacer el estudiante. El administrador crea y consulta; el estudiante elige y consulta. Esta separación protege el orden del sistema y reduce errores, porque cada acción se ejecuta desde el perfil que corresponde. En términos prácticos, esto evita que un estudiante modifique la oferta académica y refuerza la idea de que la matrícula es una decisión personal dentro de un marco definido por la institución.

La validación de datos: sostiene la calidad de la información. Pedir que la contraseña se escriba dos veces reduce equivocaciones. Limitar el número de cursos evita asignaciones excesivas. Mostrar únicamente cursos activos impide elegir opciones no disponibles. Confirmar cada guardado con un mensaje claro genera confianza en quien usa el sistema. Estas pequeñas reglas, aplicadas de forma constante, hacen que el conjunto se sienta estable y predecible.

Java: es un lenguaje y una plataforma de programación pensada para que el mismo programa funcione en distintos sistemas operativos sin cambios. Resulta útil en este proyecto porque permite crear una aplicación de escritorio que se ejecuta tanto en Windows como en Linux, ofrece bibliotecas listas para manejar ventanas y archivos, y facilita la distribución en un único paquete que la persona usuaria puede abrir sin procesos complicados.

“Java es un lenguaje de programación que se utiliza para la creación de aplicaciones informáticas, ha sido diseñado a modo de eliminar las complejidades de otros lenguajes como C y C++.” (GONZALEZ, 2009, p. 14)

El JDK (Java Development Kit): se describe como el conjunto de herramientas necesarias para trabajar con Java. Incluye lo que se necesita para compilar, ejecutar y empaquetar la aplicación. En este proyecto es un requisito, porque sin el JDK instalado el sistema no puede abrir el programa ni usar el archivo que se entrega para ejecución.

“El Java Development Kit o JDK, se refiere a la edición de Java SE, mientras que otros kits se conocen como "SDK", un término genérico para "kit de desarrollo de software." (Corcuera, 2017, p. 12)

Un archivo JAR: se entiende como un paquete comprimido que contiene el programa listo para usarse, junto con los recursos que necesita. En términos prácticos, reúne las clases compiladas y los archivos de apoyo para que la aplicación pueda abrirse con un doble clic en Windows o con el comando correspondiente en Linux. En este proyecto, el archivo CursosAcademicosUNED.jar permite distribuir y ejecutar la aplicación de forma sencilla en ambos sistemas.

“Jar: Un archivo JAR (por sus siglas en inglés, Java Archive) es un tipo de archivo que permite ejecutar aplicaciones y herramientas escritas en el lenguaje Java. Las siglas están deliberadamente escogidas para que coincidan con la palabra inglesa "jar" (tarro).” (Arboleda, 2023, p. 10)

Desarrollo

Descripción del programa

El programa actual de Cursos Académicos UNED se crea como una simulación de una pequeña universidad. Su objetivo es mostrar, de forma clara, cómo una persona se registra como estudiante, cómo se ingresa al sistema con una verificación adicional, cómo se crean los cursos que estarán disponibles y cómo cada estudiante elige sus propios cursos, tal y como ocurre en un proceso de matrícula sencillo. La experiencia inicia en una ventana principal con dos caminos visibles: registrarse o iniciar sesión. Si la persona elige registrarse, el sistema abre una ventana dedicada donde se solicitan todos los datos necesarios para identificar al estudiante y poder contactar con él: número de identificación, nombre y apellidos, fecha de nacimiento, correo, teléfono, una contraseña que se confirma al escribirla por segunda vez, además de una pregunta de seguridad secreta, simulando un doble factor de seguridad. Al guardar, el programa revisa que no falte nada, almacena la información y confirma el éxito del registro.

Cuando la persona decide iniciar sesión, vuelve a la ventana principal y selecciona el botón que lo lleva a la ventana de login y ahí escribe su usuario y su contraseña y también el sistema solicita responder una pregunta de seguridad. Esta doble verificación se mantiene igual para cualquier persona usuaria estudiante, el cual es la respuesta a cuando se registró

de cuál es su color favorito. A partir de aquí el comportamiento cambia según quién haya ingresado.

Si quien ingresa es el administrador, con el usuario y la clave establecidos en el código ("admin" y "admin"), el sistema dirige a una ventana de trabajo que reúne las opciones pensadas para su rol. El administrador puede crear cursos nuevos. Para cada curso define su código, su nombre y su estado, que puede ser activo o inactivo. Esta decisión de estado tiene efecto directo en el resto del sistema, porque solo los cursos activos estarán disponibles para que los estudiantes puedan elegirlos. El administrador también puede consultar la información registrada de los estudiantes y ver, en la misma pantalla, los cursos que cada uno tiene asignados. En cambio, la opción de asignar cursos permanece bloqueada para él. Esta limitación es intencional y busca reproducir la realidad académica: el administrador crea y gestiona la oferta, pero no decide por los estudiantes cuáles cursos llevarán.

Si quien ingresa es un estudiante ya registrado, el sistema muestra el nombre de quién inicio sesión. La misma ventana de trabajo con un conjunto de acciones adaptado a su rol. En este caso, la persona no tiene acceso a la creación de cursos, porque esa tarea pertenece al ámbito administrativo. Lo que sí puede hacer es elegir los cursos que desea llevar. Para ello, el programa presenta la lista de cursos activos y permite seleccionar y agregar hasta un límite de 4 cursos por estudiante. Si intenta superar ese límite el sistema manda un aviso. Tampoco puede escoger un curso inactivo ya que no salen en la lista del combobox, Una vez confirmada la selección, el programa guarda la asignación y comunica el resultado con un mensaje claro. Además, el estudiante puede utilizar la opción de búsqueda para consultar su información o revisar la de otros estudiantes, junto con los cursos asignados, tal como se haría al verificar una matrícula. Lo cual simula también el entorno estudiantil de la UNED donde se puede acceder a un apartado para ver los participantes y así ver a los demás estudiantes y datos relacionados.

En ambos perfiles, la navegación se mantiene simple y predecible. Cada ventana usa títulos y rótulos directos, con botones que realizan exactamente lo que anuncian: guardar para almacenar, limpiar para empezar de nuevo y volver para regresar al menú. El sistema

guarda los registros en carpetas del propio proyecto y en archivos .TXT para mantener los datos a pesar de que el sistema se cierre y se vuelva abrir. Con este diseño, la aplicación cumple con su propósito: mostrar, de forma entendible, un proceso completo que incluye registro, ingreso con verificación adicional, creación de cursos por parte del administrador y elección responsable de cursos por parte del estudiante, todo dentro de un entorno que separa funciones según el rol y que confirma cada acción para dar seguridad durante el uso.

Como funciona (Guía)

(visión del programa y del código)

El programa funciona por medio de ventanas que guían cada paso. La ventana principal abre el camino para registrarse o iniciar sesión y, a partir de ahí, conduce a la “aula virtual” donde se muestran las opciones según el tipo de usuario. Detrás de estas pantallas, el proyecto está ordenado en capas sencillas: acceso a datos, entidades, lógica de negocio, ventanas. Además, cuenta con una de utilidades para manejar un fondo para mejorar la parte estética, además de una para imágenes y poder almacenar las mismas.

El guardado de información se realiza en archivos .txt dentro de carpetas del propio proyecto. Cada estudiante se guarda en un archivo cuyo nombre corresponde a su número de identificación; esto lo maneja la clase `ArchivoEstudiante.java` en la carpeta `AccesoADatos`. Los cursos se guardan con `ArchivoCurso`. y las asignaciones estudiante–curso se conservan con `ArchivoAsignacion.java`. Así, cuando la aplicación necesita mostrar un estudiante, un curso o sus asignaciones, abre el archivo que corresponde y lee los datos desde ahí.

La ventana de registro de estudiantes escribe los datos y pide confirmación de la contraseña y la respuesta a la pregunta de seguridad. Al guardar, llama a la lógica de negocio (`EstudianteLN.java`) para revisar que no falte nada y, si todo está correcto, esta lógica pide a la capa de acceso a datos que cree el archivo del estudiante. El mismo recorrido se aplica

a los cursos: la ventana de cursos envía los datos a `CursoLN.java`, y luego esa lógica solicita a `ArchivoCurso.java` que los guarde. El estado del curso (activo o inactivo) se conserva junto con el curso y después sirve para filtrar lo que se mostrará al elegir materias.

El ingreso se controla desde la ventana Login. Cuando se trata del administrador, las credenciales fijas (“admin” y “admin”) están resueltas en `AccesoADatos` dentro de `AdministradorAD`. Cuando se trata de un estudiante, la verificación compara usuario, contraseña y la respuesta a la pregunta de seguridad con lo que está guardado en su archivo. Si es correcto, se crea una sesión con `SesionEstudiante` para recordar quién ha ingresado y mostrar su nombre en la ventana Aulavirtual

La elección de cursos por parte del estudiante se maneja con la ventana `AsignacionCursos.java`. Antes de guardar, la lógica `AsignacionLN.java` revisa dos reglas: que el curso esté activo y que el total asignado no pase de cuatro. Si las reglas se cumplen, la asignación se guarda con `ArchivoAsignacion.java`. De esta manera, el límite y la disponibilidad no se dejan a la ventana, sino a una parte del código dedicada a validar.

La consulta de estudiantes se realiza en `BuscarEstudiante.java`. Esta ventana pide a la lógica que lea los archivos según el criterio indicado y luego muestra, en una sola vista, los datos del estudiante junto con la lista de cursos que tiene asignados.

Para mantener una presentación uniforme, `UtilidadesGUI.java` centraliza detalles de las ventanas, como los fondos de pantalla que se guardan en la carpeta imágenes. Con esto, todas las pantallas cargan su aspecto de la misma forma y no se repite código.

En resumen, el programa se mueve por ventanas claras mientras el código separa responsabilidades: las ventanas muestran y piden datos, la lógica valida y decide, y el acceso a datos guarda y lee archivos de texto en carpetas organizadas. Gracias a esta separación, el funcionamiento es estable: lo que se escribe queda guardado, lo que se consulta se recupera desde los archivos correctos y cada regla como el máximo de cuatro

cursos y la visibilidad de cursos activos se cumple siempre, sin depender de la memoria del usuario.

Guía de instalación y uso de la aplicación

El programa Cursos Académicos UNED se entrega con varias formas de ejecución, de modo que la persona usuaria pueda elegir la que mejor se ajuste a sus necesidades. En todos los casos, es necesario tener instalado Java Development Kit (JDK) versión 8 o superior para poder ejecutarlo. Este requisito aplica tanto en sistemas operativos Windows como en Linux.

1. Apertura y edición del código fuente en NetBeans

Si se desea abrir el programa para revisarlo o modificarlo, se debe utilizar la carpeta Archivos del código fuente NetBeans.

Esta carpeta contiene todo el proyecto con su estructura original, listo para ser cargado en el entorno de desarrollo NetBeans. Desde ahí se puede compilar, ejecutar y realizar cambios en el código.

2. Ejecución directa mediante archivo JAR

Si solo se desea usar el programa sin modificarlo, la forma más rápida es desde la carpeta Archivo ejecutable directamente .JAR.

Dentro de esta carpeta se encuentra el archivo CursosAcademicosUNED.jar, que es la versión compilada del programa.

Es importante copiar toda la carpeta completa, incluyendo las subcarpetas:

asignaciones_estudiantes (.txt)

registros_cursos (.txt)

registros_estudiantes (.txt)

lib (contiene las librerías necesarias para que el programa funcione)

Esto asegura que el sistema pueda leer y guardar los datos correctamente.

Este programa fue diseñado y desarrollado para ser completamente multiplataforma, permitiendo su ejecución tanto en sistemas operativos Windows como en Linux. Para ello, basta con abrir directamente el archivo CursosAcademicosUNED.jar incluido en la carpeta correspondiente, siempre que el equipo cuente con el Java Development Kit (JDK) instalado, lo que garantiza su funcionamiento sin importar el sistema utilizado.

En Windows: basta con hacer doble clic en el archivo CursosAcademicosUNED.jar si el JDK está instalado y configurado.

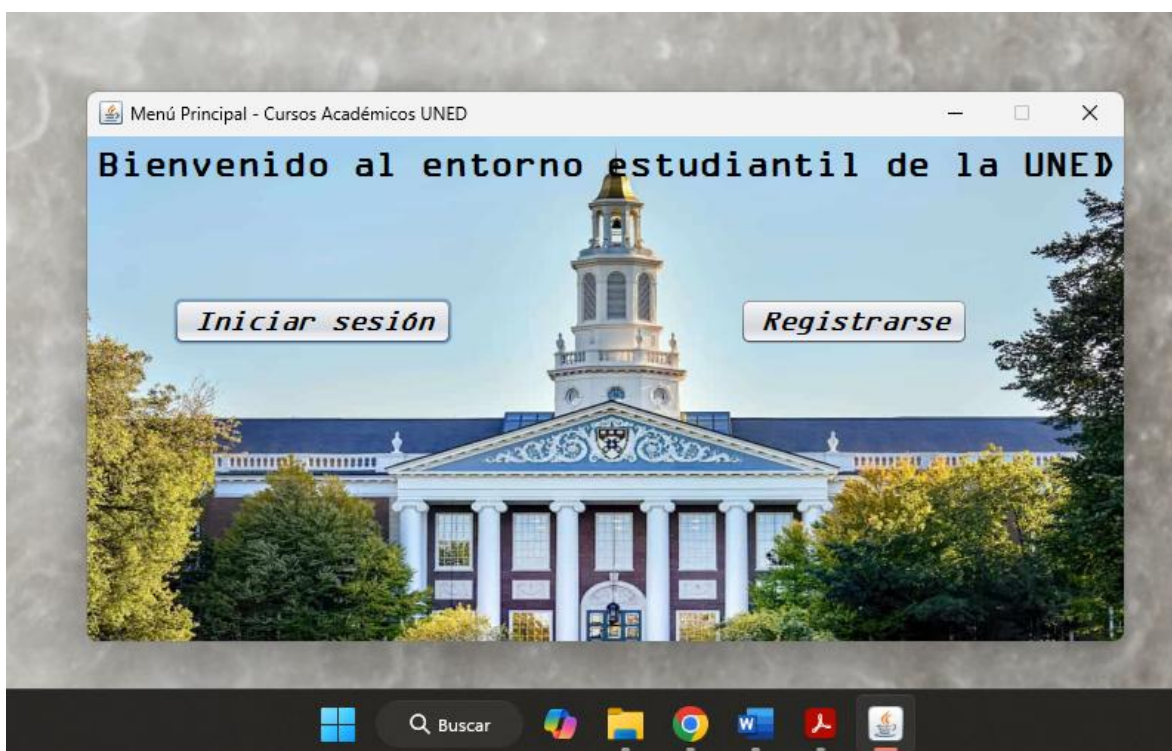


Ilustración 1, Captura Windows

Nota. Elaboración propia, donde se aprecia la ejecución del programa Cursos Académicos UNED mediante el archivo CursosAcademicosUNED.jar en Windows. Agosto 2025.

En Linux: se debe abrir una terminal en la misma carpeta donde está el archivo y ejecutar el siguiente comando:

```
java -jar CursosAcademicosUNED.jar
```



Ilustración 2, Captura Linux

Nota. Elaboración propia, donde se aprecia la ejecución del programa Cursos Académicos UNED mediante el archivo CursosAcademicosUNED.jar en Linux. Agosto 2025

3. Instalación mediante asistente “siguiente – siguiente”

Para una instalación más visual y sencilla, se incluye en la carpeta Instalador tipo siguiente siguiente un instalador que guía todo el proceso paso a paso. Al ejecutarlo, el asistente solicita confirmar la ruta de instalación y, por defecto, instala el programa en C:\Program Files. Una vez finalizado, crea automáticamente un acceso directo en el escritorio, permitiendo iniciar la aplicación con un solo clic, sin necesidad de buscar carpetas, manipular archivos o ejecutar comandos en la terminal. Esta modalidad es especialmente útil para quienes no desean configurar manualmente el entorno de ejecución, ya que todo

queda listo de forma inmediata, cabe recalcar que este instalador es exclusivo para Windows.

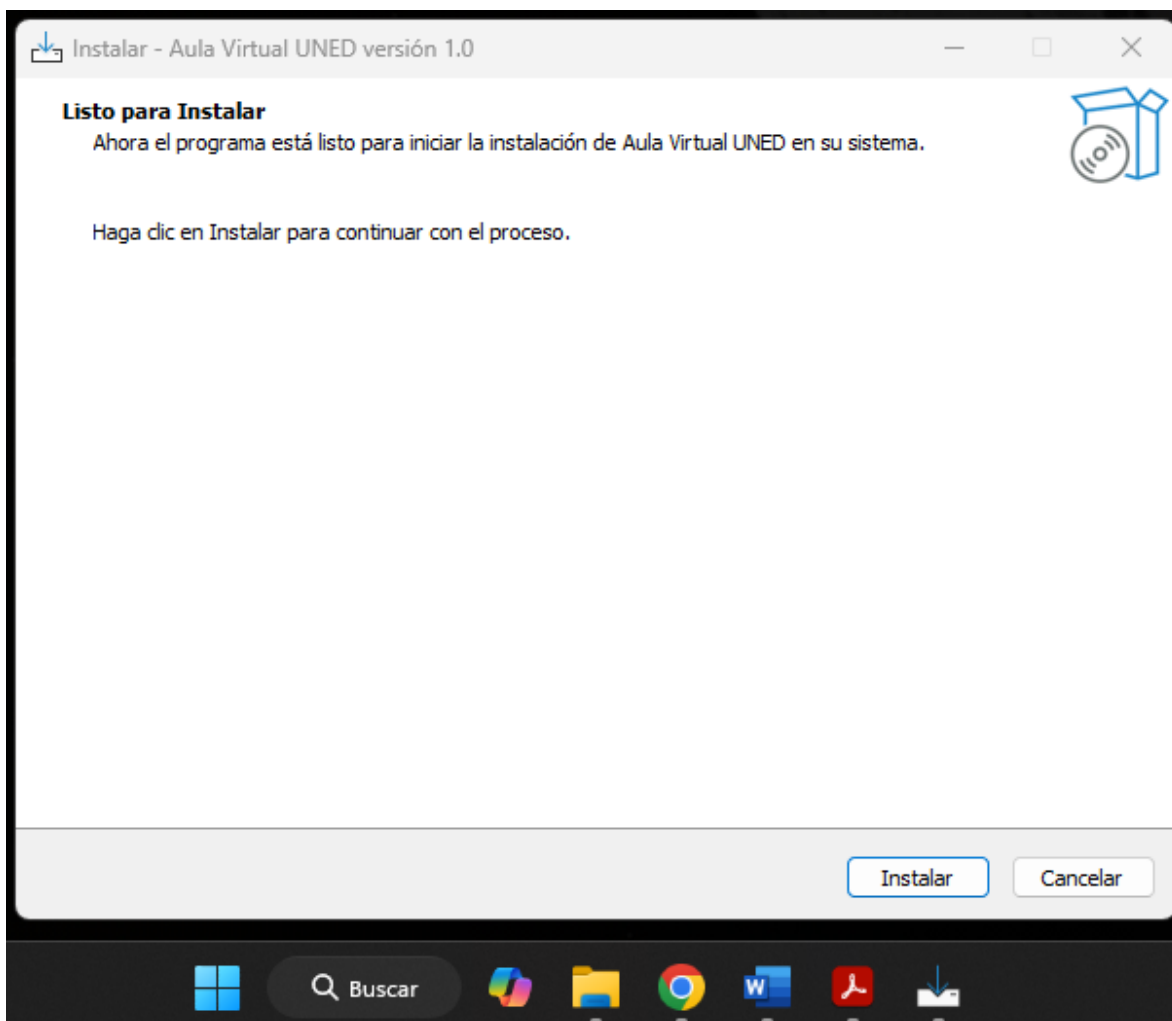


Ilustración 3, Captura instalación

Nota. Elaboración propia, donde se aprecia la ejecución del instalador generado para el programa de Aula Virtual UNED versión 1.0. Agosto 2025

En resumen, general, la aplicación puede ser utilizada como proyecto editable en NetBeans, ejecutada directamente como archivo JAR en cualquier sistema con JDK instalado, o instalada mediante un asistente gráfico que simplifica el acceso desde el escritorio. Esta versatilidad permite que el programa sea verdaderamente multiplataforma y accesible para distintos tipos de usuario, desde quienes desean modificar el código hasta quienes únicamente buscan ejecutar el sistema de forma práctica y rápida.

Limitaciones

El programa, aunque cumple con los objetivos planteados, presenta algunas limitaciones propias de su diseño y del enfoque utilizado. Actualmente, el sistema almacena la información en archivos de texto .txt, ubicados en carpetas del proyecto. Esto facilita su ejecución sin necesidad de servidores o configuraciones avanzadas, pero también reduce la seguridad, ya que estos archivos pueden ser abiertos y modificados manualmente con editores sencillos como el Bloc de notas. Una alternativa más robusta sería vincular la aplicación a una base de datos, incluso gratuita como las ofrecidas por servicios como GearHost, lo que permitiría un manejo más seguro y ordenado de la información.

El doble factor de seguridad que utiliza la aplicación se basa en una pregunta secreta con respuesta definida por el usuario en el momento del registro. Aunque este método añade una capa de protección, podría reforzarse empleando mecanismos más avanzados, como el uso de aplicaciones de autenticación como, por ejemplo, Google Autenticador o el envío de códigos temporales al correo electrónico o al teléfono del usuario.

En su estado actual, el administrador del sistema está definido directamente en el código, con credenciales fijas que no pueden modificarse desde la interfaz. Esto facilita las pruebas, pero no es lo más seguro, ya que en un entorno real sería preferible que el usuario administrador se registrara o que sus credenciales se almacenaran en un lugar seguro y cifrado fuera del código fuente.

Otro aspecto a considerar es que, para poder ejecutar la aplicación, es obligatorio que el equipo tenga instalado el Java Development Kit (JDK), lo que puede ser una barrera para usuarios con menos conocimientos técnicos o sin permisos para instalar software en sus equipos.

En cuanto a las funciones, existen posibilidades de mejora. Por ejemplo, permitir que el administrador edite los datos de un curso después de crearlo, de manera que pueda reflejar cambios como un retiro justificado por parte de un estudiante. También sería recomendable

cifrar las contraseñas guardadas, de forma que no se almacenen en texto plano en los archivos del sistema.

Finalmente, el programa no incluye opciones de respaldo automático de datos ni registros de actividad (logs) que permitan auditar acciones realizadas por los usuarios. Incluir estas funcionalidades aportaría mayor control y trazabilidad, especialmente si se pensara en ampliar el uso del sistema a un entorno real con más usuarios y datos críticos.

Conclusiones

El desarrollo de este proyecto permitió evidenciar, de forma práctica, las posibilidades y retos que presentan los sistemas operativos al ejecutar una misma aplicación en entornos distintos. La ejecución y prueba del programa en Windows y Linux confirmó que, con el uso de tecnologías adecuadas como Java, es posible garantizar un comportamiento estable y coherente en ambos sistemas. En Windows, la ejecución del archivo JAR se realiza de manera directa mediante doble clic, mientras que en Linux es necesario recurrir a la terminal, manteniendo la misma funcionalidad y estructura visual en ambos casos. Esta experiencia de uso demostró que las diferencias entre los sistemas operativos no impidieron el correcto funcionamiento del programa, pero sí influyeron en la forma de ejecutarlo y en los requisitos previos, como la instalación del JDK.

En cuanto a la parte práctica, se implementó un proceso de autenticación con doble factor, se desarrolló un registro de estudiantes con datos completos, se creó un módulo para el registro de cursos con control de estado activo o inactivo, se estableció un sistema de asignación de cursos con un límite máximo y se añadió la opción de búsqueda para consultar información de estudiantes y sus cursos asignados. Que simulan de forma fiel el funcionamiento básico de un sistema académico.

La comparación entre aplicaciones multiplataforma y nativas se refleja en la elección de Java como tecnología base. Una aplicación nativa está desarrollada específicamente para un sistema operativo, lo que puede ofrecer una integración más profunda con sus características, pero limita su uso a ese entorno. En contraste, la aplicación creada en este proyecto es multiplataforma: un único archivo JAR puede ejecutarse en diferentes sistemas sin necesidad de reescribir el código, siempre que se cumpla el requisito del JDK. Esta portabilidad es una ventaja importantísima, especialmente en entornos académicos donde conviven equipos con distintos sistemas operativos.

En síntesis, el proyecto permitió combinar la comprensión teórica de los sistemas operativos con la implementación de una solución práctica y funcional. El desarrollo multiplataforma es una estrategia factible para cubrir distintos entornos de trabajo, siempre que se planifique desde el inicio con una tecnología que lo soporte. Asimismo, el proceso puso en evidencia

áreas de mejora para futuras versiones, como el fortalecimiento de la seguridad, el uso de bases de datos en lugar de archivos de texto y la incorporación de métodos de autenticación más robustos. Con todo ello, el trabajo realizado no solo cumple con los requisitos académicos, sino que también deja una base sólida para seguir evolucionando el sistema.

Bibliografía

Arboleda, S. G. (2023). *bibliotecadigital.udea.edu.co*. Obtenido de Universidad de Antioquia: <https://bibliotecadigital.udea.edu.co/entities/publication/2290da0f-b844-465f-8206-fe47f3f0cacf>

Corcuera, P. (2017). *unican.es*. Obtenido de Universidad de Cantabria: https://personales.unican.es/corcuerp/java/slides/intro_tecjava.pdf

Deitel, P., & Harvey Deitel. (2021). *Cómo programar en Java*. México: Pearson.

GONZALEZ, Y. A. (2009). *Universidad de las Ciencias Informáticas*. Obtenido de uci.cu: https://repositorio.uci.cu/bitstream/ident/TD_2336_09/1/TD_2336_09.pdf

Reyes Riveros, A. J., Mendoza de los Santos, A., & Salinas Meza, J. E. (marzo agosto de 2023). *Universidad La Salle*. Obtenido de redalyc.org: <https://www.redalyc.org/journal/6738/673874721006/673874721006.pdf>